# ZCrypt

*A Post-Quantum Shielded Ledger*
*with Whole-Pool Anonymity*

Echelon Labs LLC

Christian Papathanasiou (founder@zcrypt.network)

*October 2026*

## Abstract

ZCrypt is a proof-of-stake ledger with one shielded pool and no transparent transfers. Every unit of value, native or otherwise, lives in that pool as a hiding commitment. Pool coins move by one shielded transfer, which publishes a pair of nullifiers, a pair of fresh commitments, two encrypted notes, a fee, an anchor, and a proof. Consensus also accepts a closed set of transparent operations around that transfer: bond and unbond, delegation, reward claims, asset registration, authority rotation, and escrows. The proof is a STARK: hash functions all the way down, no trusted setup, and nothing in it that a quantum computer is expected to break. A spend proves membership in the global commitment tree with an in-circuit Merkle path, so the anonymity set is the whole pool rather than a handful of sampled decoys, and a one-input spend is padded with a value-zero phantom so that even the number of inputs is hidden. Ownership is a hash preimage proven inside the circuit; there are no signatures in the value layer. Notes are encrypted to ML-KEM-768 keys, which means ciphertexts recorded today stay closed against a future quantum adversary. Every rule the circuit enforces also exists as a seven-instruction bytecode program that is tested differentially against the circuit and serves as the specification. Consensus is BFT with instant finality. Issuance follows a fixed 21-million schedule committed in the state root. That commitment shows the schedule was followed. It does not show that the pool contains only the coins the schedule created. Registered assets share the pool with hidden type identifiers and are issued under rotatable $M$-of-$N$ ML-DSA authorities. Bitcoin moves in and out through SPV-verified atomic swaps. A mint requires proof that BTC was locked to a secp256k1 reserve key, and that key can spend the locked coins; the shipped wallet holds it as one operator key. The fee on a transfer or a swap lock is a fixed 0.01 ZCRYPT and is burned. A redemption burn-lock pays its fee in the asset being redeemed.


## 1 Introduction

Bitcoin solved double-spending by publishing everything [1], and the bill for that decision keeps coming due. A payment on a transparent ledger is a permanent, globally replicated record of who paid whom, how much, and when, and anyone with a block explorer can read it. The second cost is fungibility. Coins acquire histories, histories acquire reputations, and exchanges refuse coins that were "tainted" by someone the current holder has never met. Cash has neither problem. A banknote does not carry a ledger.

The privacy designs built since then each give something up. Ring signatures [2] hide a spend among a small set of sampled decoys, and the anonymity that provides erodes under intersection analysis as rings overlap. Shielded pools [3] hide much more, using elliptic-curve SNARKs; the trusted setup those originally needed was later removed [7], and there are now multi-asset pools with in-pool exchange [8, 9]. But all of them rest on discrete-logarithm assumptions, which are exactly the assumptions a quantum computer breaks, and that is an awkward foundation for money that is supposed to outlive the hardware assumptions of the people who designed it. Most of them also keep a transparent ledger next to the private one. In practice most value stays on the transparent side, and the pool's anonymity set is smaller than it looks.

Zcash Orchard proves in-circuit membership against the whole note-commitment tree [4]. Monero's FCMP++ replaces the sampled ring with a membership proof over the full output set [5]. Neptune Cash is a live proof-of-work chain whose transactions are transparent STARKs over a hash-only virtual machine, with notes encrypted under lattice keys [6]. ZCrypt is a proof-of-stake pool with no transparent transfer beside it, a fixed native-coin gas burn, and an in-consensus Bitcoin SPV mint. The limits that remain are structural and are stated where they bind: the sentinel is per consensus key and can be met by splitting stake, reserve custody and the sweep sit with a secp256k1 key, zero knowledge of the proofs is unproved, the AIR is not machine-checked, and a transfer is about two megabytes.

The design rests on four decisions.

1. **One pool, one path.** There is no transparent transfer. A coin exists only as a hiding commitment in one global pool, and pool coins move by a shielded spend. Bonding, claims, registration, and escrows are separate transparent operations (§7). Nobody can opt out of privacy on a transfer, so nobody fragments the anonymity set.
2. **Whole-pool anonymity.** A spend proves, inside the circuit, a Merkle path from the spent commitment to a recent root of the global tree. The anonymity set of every spend is every coin ever created.
3. **Post-quantum on the spend path.** The proof system is a hash-based STARK with no setup. Spend authority is a hash preimage. Note encryption is a lattice KEM. No discrete-log or pairing assumption stands between an attacker and a pool coin's spend. The Bitcoin bridge is the exception that consensus itself enforces: a secp256k1 signature chooses the recipient of a zBTC mint and rotates the reserve key. Those checks are ZCrypt's rule. Bitcoin does not perform them.
4. **Transparent rules, private state.** The ledger hides who paid whom and how much. It does not hide the rules. Transparent operations have a bytecode specification that consensus runs. The shielded transfer has a bytecode specification too, but consensus does not run it: consensus checks the constraint system, the AIR. Total issuance is a function of block height committed in the state root. That

commitment shows the schedule was followed. It does not show that the pool contains only the coins the schedule created.

The rest of the paper describes the system as built: the cryptographic foundations (§3), coins and addresses (§4), the shielded transfer (§5), the auditability VM (§6), consensus and state (§7), money and staking (§8), multi-asset support and issuance (§9), the Bitcoin bridge and in-pool swaps (§10, §11), the wallet (§12), what an observer sees and what an attacker would need (§13, §14), and two ways the system could be deployed (§15).

# 2 System Overview

The network is a proof-of-stake replicated state machine with instant finality (§7). The committed state is small and its shape is entirely public. It consists of an append-only Merkle tree of coin commitments, one leaf per coin ever created; a sparse Merkle tree of nullifiers, one entry per coin ever spent; and the staking ledger, the asset registry, and an escrow book for swaps. Hash those roots together with the Bitcoin header-chain digest and the immature-mint queue from the bridge (§10), the bootstrap sentinel record (§7.4), and the recent anchor-root history (§5), and the result is the application state root that consensus agrees on every block. Nine components.

The wallet does most of the work. To pay, it picks its own coins, builds a zero-knowledge proof locally, and broadcasts a transaction that contains the proof and its public inputs and nothing else. Validators check the proof, check that the nullifiers are fresh, append the new commitments, and at no point learn an amount, an asset type, or a participant. To receive, a wallet trial-decrypts the encrypted notes attached to on-chain transactions; whichever notes open under its decapsulation key are payments to it.

Everything that moves value, whether a payment, a gas burn, a stake bond, a bridge deposit or redemption, or a reward claim, ends up as coins entering or leaving this one pool.

# 3 Cryptographic Foundations

## 3.1 Field and Hash: Poseidon2 over BabyBear

Every in-circuit object (commitment, nullifier, public key, Merkle node) is computed with one algebraic hash, the Poseidon2 permutation [10] over the BabyBear prime field $p = 15 \cdot 2^{27} + 1 = 2013265921$. The permutation has width 16, an $x^7$ S-box, 8 full rounds and 13 partial rounds. The digest is a feed-forward compression. The 16-element input is permuted once, and the digest is the first 8 lanes of $P(x) + x$, written $\mathrm{Trunc}_8(P(x) + x)$. A constant role tag occupies the last lane, so a commitment, a nullifier, a public key, and the secret coin tag are different inputs. Merkle nodes use the same compression, one child in lanes 0–7 and the other in lanes 8–15.

In the ideal-permutation model this is a compression function on an 8-limb digest. A preimage search is about $2^{248}$ permutations, and a collision search is about $2^{124}$. Those figures are the binding of a commitment, a nullifier, a public key, and a tree node alike. They are a model of the compression, not a proved lower bound on Poseidon2. Spend authority and the burn address are preimages with lanes pinned (§4.4, §5.7). A public key hashes eight secret limbs, then seven zeros and the role tag, so finding a secret for a published key, or any secret for the burn key, is an eight-lane search. Grover's algorithm takes a square root of an unstructured search, and an eight-lane preimage still has room for it under that model.

The reason for an algebraic hash is that the same function has to be cheap both natively and inside the proof. A Poseidon2 permutation costs 298 trace columns in the circuit. SHA-256 would cost tens of thousands.

Outside the circuit, byte-level hashing uses SHA3-256 [11] (state roots, transaction binding, key derivation tags, escrow identifiers), always with an 8-byte ASCII domain tag and length-prefixed fields. The FRI commitment layer inside the proof system uses Keccak. Nowhere in the value layer does anything depend on discrete logarithms being hard.

## 3.2 The Proof System: a Hiding STARK

Transfer proofs are STARKs [12] in the AIR/FRI style [13], produced and verified with Plonky3 [15]. The arithmetization is a two-dimensional execution trace over BabyBear with constraints enforced on every row. Low-degree testing is FRI with a blowup factor of 4, folding arity up to 4, and a final polynomial of length 2. Fiat–Shamir challenges come from a degree-5 extension of BabyBear, about $2^{155}$ elements. A degree-4 extension would put the out-of-domain terms under 128 bits; degree 5 lifts that ceiling above the Johnson query term below, even though the base field is small. It is not a 244-bit soundness result.

The parameters are the same in every build: 112 FRI queries at rate $2^{-2}$, and 20 bits of proof-of-work grinding on both the commitment phase and the query phase. There is no lower-security variant. The implementation's own conjectured list-decoding figure is $2 \times 112 + 20 = 244$ bits, which counts the query-phase grinding once, the same formula Plonky3 reports. That figure assumes proximity gaps up to the capacity of the code. The up-to-capacity form of that conjecture is false [14]. The Johnson bound, the estimate current analyses treat as proven, gives about one bit per query at this rate: $(2 - 1) \times 112 + 20 = 132$ bits, before degree and batching losses. Those losses sit under the degree-5 extension, about $2^{155}$ elements, and they do not restore 244. The 112 queries are also why each proof is about a megabyte. The numbers are compile-time constants. The prover

asserts at configuration time that the Johnson figure is at least 100 bits, and a node refuses to start on a real-value network below 128. Both floors are checks of the Johnson bound. The 244-bit number is the list-decoding conjecture the implementation also reports. It is not the floor. 132 passes both floors. It is not a tighter estimate than the formula above.

Zero-knowledge is provided by the polynomial commitment layer rather than by the AIR, and it is not proved. Plonky3's hiding commitment doubles the trace: the production trace is 128 rows, one random row is interleaved with each of them, four random codewords blind the quotient, and every Merkle leaf carries a salt drawn from a ChaCha20 CSPRNG seeded by the operating system. Two proofs of the same statement differ in their bytes, which shows that this seed is fresh and does not show that the witness is hidden. A count of the 112 FRI queries plus two degree-5 out-of-domain openings comes to 122 and would leave six rows against the 128 random rows. That count leaves out the sibling values a round reveals when it folds at arity up to 4, as many as three further openings of the same codeword per query. Those siblings belong in the budget, and there is no simulation argument that what remains is enough. A stock transparent STARK exposes raw trace values in its Merkle openings, and the privacy claims in §13 rest on this layer doing otherwise. If it does not, every proof already published gives up its witness to anyone who kept a copy, and a later upgrade does not take those proofs back. There is no trusted setup of any kind; the only preprocessing is a public matrix of selector columns that prover and verifier both derive from the circuit description.

### 3.3 Note Encryption: ML-KEM-768

A payment has to get the coin's opening (value, blind, asset identifier) to the recipient and to nobody else. Each output slot carries an encrypted note. The sender encapsulates to the recipient's ML-KEM-768 [16] public key, expands the shared secret with HKDF-SHA256 [17] with the KEM ciphertext and the recipient's encapsulation key bound into the derivation, and encrypts the note under ChaCha20-Poly1305 [18]. The plaintext is 48 bytes, padded to a fixed 256, so every note ciphertext on chain is the same size whatever it contains.

Recipients scan by trial decryption. A note either authenticates under the wallet's decapsulation key or it is noise. Because ML-KEM is a lattice scheme standardized for post-quantum confidentiality, notes harvested from the chain today cannot be opened later by a quantum computer. ECDH-based note encryption cannot make that promise.

Output slots that should not carry a real note, for instance when the payment is delivered out of band (§4.3), carry a cover note instead: a correctly shaped ciphertext encapsulated to a key that is then thrown away. Nobody watching the chain can tell the two apart.

## 4 Coins, Keys, and Addresses

### 4.1 The Coin

A coin is four things: a 52-bit value $v$ in atomic units, a roughly 124-bit blinding factor $b$ (four field limbs), a roughly 248-bit spend secret $s$ (eight field limbs), and an asset identifier $a$ that occupies one BabyBear lane. The native coin, ZCRYPT, is identifier 0. The lane is about 31 bits. A 64-bit identifier does not fit, and the registry reduces every ticker hash into that lane before it stores it (§9). The only thing derived from the coin that appears on chain when it is created is the commitment

$$C = \mathsf{P2}\big(v_{\mathrm{lo}}, v_{\mathrm{hi}},\, b,\, pk,\, a\big), \qquad pk = \mathsf{P2}(s),$$

with $\mathsf{P2}$ the feed-forward compression of §3.1 and the value split into two 26-bit limbs. The circuit puts the recipient's public key into that hash, so the creator names a recipient. A second opening is a collision of that compression, the search in §3.1. Value, owner and asset type are what the hash is computed on. How well it hides them is the usual hiding of a blinding factor inside a permutation, which is not the same property as binding.

When the coin is spent a second object appears, the nullifier

$$nf = \mathsf{P2}\big(s,\, v_{\mathrm{lo}}, v_{\mathrm{hi}},\, b,\, a\big).$$

The leaf position is not an input. The same opening always produces the same nullifier, and producing it requires the spend secret, so spending one coin twice collides in the public set and the second spend is rejected. Creating that opening twice is rejected at the tree. Consensus refuses a commitment that is already a leaf, including two copies inside one transaction, and a snapshot whose leaves repeat does not load. Zcash called a repeated opening Faerie Gold and closed it by putting the position into the nullifier [3]. This nullifier has no free lane for a position, so the tree's uniqueness rule is what keeps one opening from occupying two leaves. Reward claims, bridge-fee claims, and escrow settlements derive their blinds from a public index or from the escrow identifier, and that index advances on every coin. Nobody without the spend secret can connect a nullifier to a commitment.

Values are capped at $2^{52} - 1$ atomic units per coin. With six decimals that is about 4.5 billion ZCRYPT, far more than exist, so the cap never binds.

### 4.2 Key Hierarchy

An account comes from a 24-word BIP-39 mnemonic [19] carrying 256 bits of entropy. The mnemonic seed is domain-hashed into the spend secret $s$ and into a deterministic ML-KEM-768 keypair for notes; both come back from the phrase alone. The spend public key $pk = \mathsf{P2}(s)$ is a hash image, not a group element.

### 4.3 Addresses and Payment Instruments

An address is a kind tag, the 64-byte spend public key, and the 1184-byte ML-KEM encapsulation key, encoded

in bech32m [20] with human-readable part zc. That is about 1.2 kB, which is long. The bech32m implementation lifts the 1023-character limit; the checksum's formal error-detection guarantee holds inside that limit, and beyond it detection is probabilistic at $1-2^{-30}$. Three payment instruments share the machinery.

- **Normal address.** The account's primary address. Payments to it carry a real encrypted note on chain and are found by scanning. Recoverable from the mnemonic.
- **Stealth address.** Structurally the same address, backed by an independent key set. Stealth subaccounts derived from the master seed and an index are what the wallet uses for change, so change outputs cannot be linked to the published address. Stealth receive addresses that a user creates on purpose use fresh random seeds; they are not derivable from the mnemonic and the account's view key cannot see them. For a stealth payment the on-chain slot carries a cover note and the real note travels out of band, as a file or a string. On the wire a stealth address looks exactly like a normal one.
- **Bearer coin.** A coin with its spend secret inside the token: 112 bytes (value, blind, spend secret, asset) in bech32m with prefix zcb. Whoever holds the string holds the value. Claiming sweeps the coin into a spend keyed to the claimant, which makes the previous holder's copy of the secret worthless.

**View keys.** An account can hand out its ML-KEM key pair without the spend secret. Whoever holds it can decrypt every incoming note to the normal address, amounts and asset types included, but cannot spend and cannot derive nullifiers, so cannot see when the funds leave. Stealth and bearer instruments are outside the view key's scope by construction. This is how an account discloses selectively, to an auditor say, without giving up custody.

### 4.4 No Signatures

There is no signature scheme in the value layer. Authority to spend is knowledge of $s$, and the circuit is where that knowledge gets proven: the spent commitment is constrained to open to $pk = \mathsf{P2}(s)$ for the $s$ the prover supplies, and the published nullifier is constrained to derive from that same $s$. Malleability inside one proof is closed off separately. Every ciphertext, commitment and fee, and the proof itself, are bound into a transaction-binding hash that is stored next to each nullifier of that proof, so those fields of that proof cannot be swapped after broadcast. A second proof packaged beside the first is bound by its own auxiliary digest. The gas proof of §5.7 commits to the transaction it pays for. The upshot for a pool coin's spend is that there is no ECDSA or EdDSA between the spend secret and an attacker. The bridge signatures of §10 are outside that sentence.

It also means there is no in-protocol multisig for user funds. If authority is knowledge of $s$, then threshold custody means splitting $s$, or the mnemonic behind it, by secret sharing outside the protocol and reconstructing it to spend. Issuance authorities are threshold-governed on chain (§9.1); individual coins are not, and institutional custody of pool coins is, for now, a key-management problem rather than a protocol feature.

## 5 The Shielded Transfer

### 5.1 Transaction Contents

A transfer publishes up to two nullifiers (one per input slot), two output commitments, two fixed-size encrypted notes (real or cover), a fee field, an anchor root, and the STARK proof. That is the whole wire format. Hidden: amounts, asset type, sender, recipient, which coins were spent, and how many (§5.6). Public: that some valid spend of pool coins happened, and the fee, which for a user transfer is a constant (§5.7).

### 5.2 The Proven Statement

The proof has 54 public field elements: the two output commitments, the fee, the two nullifiers, the anchor root, a five-limb chain-domain tag, and an eight-limb auxiliary digest. The digest binds the transaction's encrypted notes into the proof; for escrow and staking transactions it binds their plaintext parameters as well. A gas proof binds, in that same digest, a domain-separated hash of the transaction it pays for (§5.7). Against those inputs the proof says that the prover knows openings $(v_i, b_i, s_i, a)$ for each input and $(v'_j, b'_j, pk'_j, a)$ for each output such that:

1. **Membership.** Each active input's commitment is a leaf of the depth-32 commitment tree under the public anchor root, shown by an in-circuit Merkle path of Poseidon2 compressions. The path proves existence and position and reveals neither.
2. **Ownership.** Each input commitment opens to $pk_i = \mathsf{P2}(s_i)$; the prover knows the spend secret.
3. **Nullifier correctness.** Each published nullifier equals $\mathsf{P2}(s_i, v_i, b_i, a)$ for the same witness values, so a nullifier cannot be forged or detached from its coin.
4. **Output correctness.** Each public output commitment opens to its witness values with a well-formed layout.
5. **Range.** Every value is decomposed into 52 constrained bits, the fee into 26, and nothing can overflow the field.
6. **Conservation.** $\sum_i v_i = \sum_j v'_j + fee$, enforced limb by limb with a range-checked carry, so it holds over the integers and not just modulo $p$.
7. **Single asset.** Inputs and outputs all carry one asset identifier, so value cannot leak from one asset into another inside a transaction.

8. **Domain.** A hash of the chain identifier is a public input, so a proof made for one network is useless on another.

The membership proof is the centre of the privacy model. A spend hides among all $2^{32}$ leaf slots of the tree, every coin ever created, and the set grows as the system does. Nothing is sampled, so there is nothing for an intersection attack to intersect.

## 5.3 Anchors and the Root Window

The tree is append-only, so its root changes in every block that adds coins. A proof is built against whatever root the wallet last synced. Consensus accepts any anchor that is a block-boundary root from the last 1000 blocks or one of the 64 most recent intra-block roots, and it checks the anchor before it bothers verifying the proof. The window is itself part of the state root, so validators cannot disagree about which anchors are live. Wallets never race the chain, and a proof built against ancient state simply expires.

## 5.4 Tree Capacity and Generation Rollover

The membership circuit fixes the Merkle depth, so a tree holds a bounded number of leaves. At depth 32 that is about 4.29 billion. Only real output commitments take a leaf (two for the payment, two for its gas note; the value-zero phantom is an input and never enters the tree), so a depth-32 tree holds on the order of a billion transfers. Still, it is finite, and that is handled at two levels.

First, depth is a launch parameter. Each level adds one Poseidon2 compression per input to the authentication path, so depth trades directly against proof cost. Going from depth 20 to depth 32, a 4096-fold capacity increase, costs about 2.4% in proof size and roughly 30% in proving time, with verification unchanged (§6.2). A deployment picks a depth for the lifetime it expects.

Second, the reference implementation does not yet roll over. If the tree fills, the chain halts rather than dropping a coin or quietly moving it somewhere else, and continuing past that point is a coordinated upgrade. The intended mechanism is a fresh tree of the same depth, a new generation, whose root the public anchor identifies. The nullifier set would stay global, one sparse tree across all generations, so a coin could not be respent by hopping between them, and the root window would straddle the boundary so no wallet stalls. The cost would be that a spend hides among its own generation's coins rather than all of history. Recursive aggregation, which folds sealed generations into one accumulator a spend can prove against without an ever-longer path, is the way back to a single whole-history set. Both are future work. At four leaves per transfer the depth-32 tree is not going to fill soon.

## 5.5 The Nullifier Set

Spent-coin state is a sparse Merkle tree of height 256 keyed by a hash of the nullifier, with the limbs reduced to canonical field form first so that no second encoding of the same nullifier exists. The leaf value is the spending transaction's binding hash, which means two conflicting spends of one coin are distinguishable in the tree itself. Insertion is first-spend-wins, and a transaction with any nullifier already present is rejected. The tree gives compact inclusion and non-inclusion proofs, which nodes serve on request, so anyone can ask "is this nullifier spent?" and verify the answer against the registry root rather than take a node's word for it. The reference wallet answers that question from the blocks it already stores (§12).

## 5.6 Arity Hiding

A transaction always publishes exactly two nullifiers and two commitments. When a wallet spends one coin, the second input slot gets a phantom coin of value zero. Its spend secret and blind are derived by folding a domain constant and the slot index, through Poseidon2, with each real input's secret coin tag. That tag hashes the real coin's spend secret, value, blind, and asset identifier under a domain distinct from the nullifier, and it is never published. The phantom is exempt from tree membership, since a value-zero input cannot create value, but it publishes a real, unpredictable Poseidon2 nullifier like any other input, and consensus records that nullifier as spent like any other. One-input and two-input spends therefore look identical on chain. The tag binds the whole coin, so two transactions never produce the same phantom, even when a spend secret is reused. Deriving the phantom from the published nullifier would let an observer recompute the padding slot and see that the spend was short; the secret tag is what keeps that hidden. The all-zero nullifier is a reserved constant; a transaction that publishes it is rejected.

## 5.7 Gas

Every user transfer pays a fixed 0.01 ZCRYPT, and the fee is burned. Nobody receives it. Since the pool hides asset types, the fee cannot be a percentage of a hidden amount, so a transfer is really two proofs applied atomically: the main transaction, in any asset and with a zero fee field, and a gas transaction spending native coins whose first output consensus requires to be the canonical burn commitment

$$C_{\text{gas}} = \mathsf{P2}(0.01\,\text{ZCRYPT},\, b_{\text{gas}},\, pk_{\text{burn}},\, 0),$$

with the blind $b_{\text{gas}}$ revealed in the transaction so anyone can check the burn. The burn key $pk_{\text{burn}}$ is a fixed constant with no known Poseidon2 preimage; coins committed to it are unspendable and gone for good. ZCRYPT is therefore the universal gas asset. Moving any asset consumes native coin, and the asset being moved is neither taxed nor revealed. The two proofs may not share

a coin, and consensus applies them together: both must verify before either takes effect. The gas proof's auxiliary digest commits to a domain-separated hash of the main proof's binding hash, under the tag that marks a gas payment. Registration and escrow-open use the same digest over the transparent body that follows the gas prefix. The digest is never empty. A gas proof built with no envelope does not verify, and a gas proof does not verify as payment for a different transaction. Consensus checks that binding before it records either proof's nullifiers. Outputs of the pair must also be fresh commitments, distinct from each other and from every leaf already in the tree (§4).

Two of these choices deserve a word: why burn, and why fix the amount. Paying the fee to validators would mean crediting a hidden-amount transaction to a public ledger, which either reveals the fee or adds a second accounting path consensus has to audit. A fixed burn needs neither; the burn commitment is checkable from the revealed blind. A fee market would also cost privacy. Whatever fee a user picks is a public number on an otherwise uniform transaction, and wallets that pick differently become fingerprintable. One constant closes that channel. The 0.01 figure is a consensus parameter that only a coordinated upgrade can change.

The same choices have costs the constant does not pay. A valid flood cannot be outbid, because every honest transaction pays the same fee; the sender who can prove fastest fills the block. On the laptop in §6.2 a transfer takes about 0.22 s to prove, so one machine produces a few transfers a second and burns 0.01 ZCRYPT on each of them. An invalid proof is cheaper than that. It costs a validation the verification time, about 10 ms, and the bandwidth of a megabyte-scale blob, and it burns nothing if it never verifies. Proposers earn none of the fee, because the fee is burned, so declining to include a transaction costs a proposer nothing. While emission is large, staking rewards pay validators to be online. As the schedule halves away, that payment shrinks and the burned fee does not replace it. A valid flood does destroy the attacker's own coins. It is not, by itself, a reason for a proposer to prefer one transaction over another.

# 6 Transparent Rules: the Auditability VM

A privacy system asks people to trust rules they cannot watch run. This design narrows that trust to something a person can read. Every transaction type has a short bytecode program for a small VM built for the purpose, and for transparent operations that bytecode is what consensus runs. For the shielded transfer the bytecode is the specification of the statement. The AIR is what consensus enforces.

For the shielded transfer the program is seven opcodes:

CHECK_RANGE · CHECK_MEMBERSHIP · CHECK_SINGLE_ASSET · CHECK_CONSERVATION · COMPUTE_NULLIFIERS · COMPUTE_OUTPUTS · HALT

Each opcode implements one constraint family of the circuit in plain integer arithmetic over the clear witness, down to reproducing the phantom-coin padding bit for bit. The VM cannot run at verification time, because the chain never sees a witness, so it runs at the two places where witnesses exist. A differential test suite asserts VM ≡ circuit on valid and adversarial witnesses, and every wallet runs the VM after proving and refuses to broadcast if the VM and the proof disagree. If the prover is ever wrong, the wallet fails closed rather than sending.

Transparent transactions (staking, claims, asset registration, authority rotation, escrows) use a second instruction set whose opcodes check stake minimums, claimable balances, ticker validity, escrow state, timeouts, hashlock preimages, burn outputs, and threshold signatures against a registered authority. For these the VM is the gate: consensus runs the program and its answer is the accept/reject decision. An auditor can read those programs, a few dozen bytes in all, and know what those operations are allowed to do. The transfer is the operation this does not cover. Its wallet-side VM runs where the witness exists, on the prover, and a differential test compares the two on sampled witnesses. An attacker uses their own prover and never runs that check. Sampled agreement rarely finds an under-constrained column, which is the usual way a circuit fails. Reading the bytecode says what an honest proof was built to do. It does not say what a proof can get away with. That question is about the AIR, and §14 records that the AIR is not machine-checked against the bytecode.

## 6.1 Circuit Engineering

The transfer circuit exists in two differentially tested forms that prove the same statement with the same public inputs. The wide reference trace is 32 rows by 22 272 columns, all 72 Poseidon2 permutations side by side. The production tall trace is 128 rows by 2 643 columns plus 6 preprocessed selector columns, and it runs the 64 Merkle-path compressions one per row. Proof size in a STARK grows linearly with width and only logarithmically with height, which is why the tall layout produces much smaller proofs for the same statement. The wide circuit is kept as the reference the tall one is checked against; cross-verification between the two, and against the VM, is part of the test suite. These are the depth-32 dimensions from §5.4. The permutation and compression counts scale with tree depth; the tall column count does not.

### 6.2 Measured Performance

The medians below were taken on an Apple M3 Max laptop (14 cores, 36 GB), release build, at the only parameter set this binary has (112 FRI queries, §3.2), over 12 proving runs, 40 verification runs, and 2000 VM runs per configuration.

| | **2-in/2-out** | **1-in/2-out** |
|---|---|---|
| Prove (build the tx) | 107.4 ms | 109.3 ms |
| Verify (network check) | 10.4 ms | 10.4 ms |
| Transparent VM audit | 73.8 $\mu$s | 46.4 $\mu$s |
| Proof size | 1015 KB | 1015 KB |
| Verify throughput / core | ∼96/s | ∼96/s |

The identical proof sizes are arity hiding (§5.6) showing up in the numbers: a one-input spend is byte for byte the same shape as a two-input spend. A full transfer is two proofs, gas note plus main transaction (§5.7), so a wallet builds a complete private payment in about 0.22 s and a validator core admits it in about 21 ms of proof checking. That is a verification rate near 50 transfers a second per core on this laptop, if the bytes are already in hand.

Bandwidth is the limit, and it binds well below that rate. A complete transfer carries roughly 2 MB of proof. The reference image raises CometBFT's per-transaction mempool cap to 5 MB, which fits one transfer; the stock cap of about 1 MB would not. The block cap is left at CometBFT's stock value, about 21 MB, so a block holds on the order of ten transfers. At ten transfers a second each validator moves about 160 Mbit/s of proof bytes before gossip amplification. The verification rates above are not a throughput the network can carry. Proofs are checked at admission and at finalization and are not part of the committed state, which commits to trees and ledgers. Pruning the proof bytes therefore does not change the state root. It does change what a new node can check: a node that joins from a snapshot trusts that snapshot and the validators who signed the height it came from, rather than re-checking every historical proof. The proving latency, about 0.22 s on this laptop before the one-second block, is a per-payment cost. It is not a comparison with a card network's authorization rate.

## 7 Consensus and State

### 7.1 BFT Consensus

ZCrypt is an ABCI application driven by CometBFT [21]. Blocks are final when they commit, with no reorganizations and no confirmation heuristics, and the reference deployment targets a one-second block interval, with CometBFT allowed up to two seconds to commit. The application validates a transaction at mempool admission (full proof verification, anchor and nullifier checks, no state mutation) and then again, authoritatively, when the block is finalized.

All value flows through the shielded transfer. The remaining transaction types are a closed set of transparent operations: bond and unbond, delegation, reward claims, asset registration, and the escrow operations of the bridge. Anything else is rejected. There is no script layer and no general-purpose execution, so there is nothing for an attacker to program.

### 7.2 The State Root

Every block commits an application hash

$$H\big(root_{\mathrm{nf}},\ root_{\mathrm{tree}},\ root_{\mathrm{stake}},\ root_{\mathrm{assets}},\ root_{\mathrm{escrow}},\ root_{\mathrm{btc}},\ root_{\mathrm{immature}},\ root_{\mathrm{sentinel}},\ root_{\mathrm{anchors}}\big)$$

over the nullifier set, the commitment tree, the complete staking ledger (stakes, delegations, rewards, unbonding queue, tombstones, validator keys, per-key nonces), the asset registry (each ticker's supply counter, cap, authority epoch and reserve-key history), the escrow book, the Bitcoin header chain and its claimed-outpoint set (§10), the immature-mint queue, the bootstrap sentinel record (§7.4), and the window of live anchor roots. Total issuance is inside the staking digest, so "supply equals the emission schedule" is a claim anyone can check against one committed hash. Node state serializes to a compact snapshot written atomically at every commit. A node restarted from its snapshot reproduces the committed root exactly; a corrupt snapshot halts the node instead of letting it rejoin from genesis and pretend nothing happened.

### 7.3 Dynamic Validator Set

Nobody configures the validator set. It is a function of the staking ledger, with one bootstrap exception described next. Each block the application diffs the ledger's validator set before and after the block's transactions and pushes the power changes to CometBFT: a bond adds a block producer at one unit of voting power per whole staked coin, an unbond or a slash removes one, and delegation changes reweight. All of it is derived from state that sits under the state root.

### 7.4 Bootstrap Liveness Guarantor

A BFT chain cannot finalize a block until an honest supermajority of stake exists, and at genesis there is no such thing. So the genesis app_state may name one foundation validator as a sentinel. While it is active, the application pins its voting power at max(floor, 2·others+ 1), just over two thirds of the total, and the chain keeps producing final blocks before an independent validator set has formed. During bootstrap that one validator's power has nothing to do with its stake, and the network is not decentralized. The sentinel's identity and its inflated power are both committed in the state root, so at least it is not hidden.

It is meant to go away on its own. The sentinel retires to its ordinary staked power once external stake crosses

a threshold set in genesis, provided no single external validator key holds more than a third of that stake and the condition has held for a dwell window of blocks. The bound is per consensus key. Stake split across several keys, each at or under a third, still satisfies it, so one party can meet the condition by splitting. The dwell stops a spike that does not last. It does not tie those keys to one owner. The sentinel can also be retired early by a transaction signed with its own consensus key. Once it retires the set is purely stake-weighted and no node is special.

The current mechanism has two limits. The sentinel does not bond stake of its own by default, so the evidence-triggered slashing in §8 has nothing of the sentinel's to burn while it is active. And any validator with more than two thirds of the power can decline to commit the block carrying evidence against it, at the cost of halting the chain in full view. Making the sentinel post a slashable genesis bond, retiring it on tombstone, and adding a hard sunset height are on the list. The whole arrangement is a crutch with a committed, automatic end, and the retirement threshold is a launch parameter an operator sets.

# 8 Monetary System and Staking

## 8.1 Emission

Supply is fixed at 21,000,000 ZCRYPT with six decimals, at a one-second target block time. Twenty percent, 4.2M, goes to the foundation at genesis, but not as liquid coins. It vests on an on-chain schedule that is purely a function of height. From genesis, 50,000 ZCRYPT is liquid, for validator bonds and operations. At month twelve the vested total steps to 25% of the allocation, 1,050,000, and that figure already includes the 50,000. Then 87,500 per month for 36 months, through month forty-eight, adds the remaining 3,150,000. The cliff plus those months is 4.2M. Consensus rejects any foundation claim that runs ahead of the cumulative total. The other 80%, 16.8M, is the budget for validator and delegator rewards: 0.14 ZCRYPT per block, which is 140,000 atomic units, halving every 60,000,000 blocks (about 1.9 years). The integer shift reaches zero at the 18th halving, about 34 years, because 140,000 shifted right 18 times is zero. A guard at 63 halvings is present in the code and never binds. Summing the eighteen non-zero eras mints 360 ZCRYPT less than 16.8M; the clamp keeps issuance from ever going over, and that 360 is simply never minted. Rewards pay out every 100-block epoch. There is no mint authority anywhere in this path. Issuance is the vested foundation amount plus epoch rewards, and both are in the state root.

The 0.01 gas burn (§5.7) and slashing burns destroy supply permanently, so the net supply ends up strictly below 21M.

## 8.2 Bonding as Provable Burning

A validator joins by spending shielded coins into the burn key. The bond transaction has an ed25519-signed header (the consensus key that will sign blocks, the stake, a reward address, the validator's commission, and a per-key monotonic nonce, all signed with the chain identifier so the authorization works once and only on this chain) plus a shielded spend whose first output must be the revealed-value commitment of exactly the stake, bound to $pk_{\text{burn}}$. The stake provably leaves the pool.

The pledge proof is also tied to that header. Consensus key, stake, reward address, commission and nonce form an envelope that goes into the proof's auxiliary digest (§5), so someone who sees a pledge in the mempool cannot wrap it in their own signed header and claim it. Delegations bind the delegator identity, target and reward address the same way. The stake amount is public, as it has to be, but where the coins came from is not; they are ordinary anonymous pool coins. Minimum self-bond is 10,000 ZCRYPT. Delegation works the same way, for any positive amount, to any bonded validator.

## 8.3 Rewards, Unbonding, Slashing

Epoch rewards split by weight $w_v = (\text{stake}_v + \text{delegated}_v) \times \text{blocks signed}_v$, so a validator that was absent earns nothing no matter how much it staked. Each validator sets its own commission on the delegated share at bond time, in basis points, with a default of 10%, and the rest divides pro rata. Rewards accrue to a ledger keyed by reward address and are claimed with a gasless transaction that mints coinbase coins straight into the pool at commitments the claimant's wallet can derive, so even fresh rewards enter circulation as ordinary anonymous coins.

Unbonding removes the validator, cascades its delegations, and parks the stake in a queue for 10 days of block time; when it releases, the stake is a claimable balance that comes back into the pool through the same claim path. Equivocation or light-client-attack evidence from CometBFT burns 5% of the offender's own stake and 5% of each delegation, force-unbonds the remaining 95% of both through the queue, and tombstones the validator's identity for good. Delegators lose that same 5% and cannot jump ahead of it, because undelegation goes through the same 10-day queue. A client that has been offline longer than the unbonding period cannot tell a long-range rewrite, built from keys that have since unbonded, from the chain the current validator set extends. The client ships a weak-subjectivity checkpoint for that distinction: a height, the application hash, and the hash of the bonded validator set. The binary's checkpoint is the empty node at height 0, the state before genesis mints the foundation allocation, the sentinel, or zBTC. It is not the application hash InitChain returns. At the checkpoint height the state root and the validator-set hash must match, and the match is recorded. A snapshot above the checkpoint loads only with that record.

The record is not part of the state root. A launch replaces the compiled checkpoint with one inside the 10-day unbonding window. A client offline longer than that window needs a new checkpoint from a source it trusts. A restart from a snapshot inside the window still trusts that snapshot once the checkpoint has been recorded.

# 9 A Multi-Asset Shielded Pool

The pool does not care what asset a coin is. The asset identifier inside each commitment is one Baby-Bear element, the reduced form of a ticker hash, not a 64-bit string. The circuit enforces that a transaction is single-asset and conserves value, but the identifier never appears on the wire. An observer cannot tell a ZCRYPT transfer from any other, so every asset shares one anonymity set.

Assets are created in a first-come registry. A ticker of 2 to 10 alphanumerics, case-insensitive, maps by hash to that field element, with a declared decimal precision and a maximum supply the registry does not raise afterwards. Exceeding that cap by opening one commitment as a larger coin of another asset is a collision of the compression in §3.1. Registration rejects a ticker whose reduced identifier collides with one already registered, including zBTC and the native identifier 0, so a new ticker cannot be enrolled as an alias of an existing asset. The reduced space is about 31 bits. Birthday collisions make a fresh ticker un-registerable once tens of thousands of assets exist; they do not make two registered assets share a lane. Minting only happens through the escrow mechanism of §10, authorized either by the asset's registered threshold authority (§9.1) or, for zBTC, by an in-consensus Bitcoin SPV proof, and consensus caps it at the registered maximum. The per-asset supply counter is in the state root. Redemption of a bridged asset decrements that counter. The native asset is identifier 0, is never registered, and has no such counter: it never leaves the system, so nothing outside the pool can be reconciled against it (§14). The zBTC ticker is special-cased at the registration chokepoint: consensus refuses any registration of it that does not carry exactly eight decimals, the 21-million cap, and the SPV mint form of §10, so nobody gets to choose the bridge asset's parameters or its issuance rule.

## 9.1 Threshold Mint Authority

A mint authority is an $M$-of-$N$ threshold of ML-DSA-65 keys [22], the lattice signature standard that goes with the ML-KEM encryption in §3.3, so issuance is post-quantum end to end. The registry stores only a 32-byte commitment, a hash of the threshold and the sorted key set of up to 32 keys. To open a mint escrow the submitter presents the key set and at least $M$ signatures over a message that binds the exact escrow parameters (asset, value, hashlock, timeout, recipient key) under a protocol context string. The VM checks that the key set hashes to the registered commitment and that the threshold verifies. A signature therefore authorizes one specific issuance and nothing else, and replaying it collides with the deterministic escrow identifier.

Authorities rotate on chain. Each asset carries a monotonic authority epoch, committed in the state root. A rotation presents the current key set with $M$ signatures over (asset, current epoch, new commitment); on acceptance the registry swaps the commitment and bumps the epoch. Because the epoch is in the message, a rotation signature works once, and rotating back to an old key set cannot replay an old authorization either. Signers never put a secret on chain in either operation, only public keys and signatures.

A single user issuing a personal asset uses a 1-of-1 authority derived from their wallet seed. A consortium uses a real multi-party set. Same machinery either way. zBTC is the one exception: it has no threshold authority at all and is gated instead by the SPV proof of §10, so no group of signers can mint it.

Gas is always native (§5.7), so an asset transfer has a zero fee field in the main proof and burns ZCRYPT in the gas proof. The one exception is bridge redemption, where the in-transaction fee is paid in the asset being redeemed.

# 10 Bitcoin Interoperability

Bitcoin enters the pool as zBTC, a registered shielded asset with eight decimals and a 21M cap, minted and burned one for one against BTC. There is no discretionary mint authority and no price oracle behind it. Every validator verifies Bitcoin itself, through an SPV client that lives inside consensus, and mints only against a proof that real BTC was locked. The threshold authority of §9.1 is for user-issued assets; zBTC is governed by the SPV gate described here, and consensus will not register the zBTC ticker in any other form.

## 10.1 An In-Consensus Bitcoin Light Client

The chain keeps a Bitcoin block-header chain as part of its own state, digest included in the state root. Anyone can submit a batch of 80-byte headers in a relay transaction; the relayer is untrusted because each header carries its own proof of work. Validators check that a header links to its stored parent, meets the target consensus requires at its height, follows the retargeting schedule (Bitcoin's 2015-interval timespan with the fourfold clamp, off-by-one and all), and post-dates its median-time-past, and they follow the heaviest-work chain the way Bitcoin does. Mempool admission runs the same validation as a dry run, so a header that could not enter the chain also cannot occupy the bounded per-block relay budget, and that budget is only charged for batches consensus actually accepts. A flood of headers with self-declared easy difficulty gets thrown out before it costs anyone any-

thing. A production deployment pins a recent header as a checkpoint and tracks headers from there. Rewriting this chain would mean out-mining Bitcoin, and one honest relayer is enough to defeat one that withholds.

## 10.2 The SPV Mint

zBTC's registered authority is not a key-set commitment but a commitment to one Bitcoin reserve public key. To mint, a party submits an inclusion proof: the raw funding transaction plus a Merkle branch showing it sits in a block on the chain's best header chain. Consensus accepts it if the block is buried to a network-dependent minimum depth, the branch reconstructs the block's Merkle root, and the funded output pays the pay-to-witness-script-hash of exactly the swap's HTLC, meaning the shared hashlock, the pinned reserve key as claimant, the depositor's refund key, and a safe refund locktime. The locked satoshis must cover the minted base units one for one. Bitcoin's Merkle tree is ambiguous between a 64-byte transaction and an inner node. The mint check rejects a funding transaction or a coinbase whose non-witness serialization is 64 bytes, and it requires the coinbase of the same block in the same inclusion proof, with that coinbase at index 0. The branch is then a transaction, at a height the coinbase pins, on the chain's best header chain.

Binding the deposit to the reserve key stops the depositor from taking the success path. The success path of the script pays the pinned reserve key, so a deposit locked to a key the depositor also controls fails the script check. The timeout path still pays the depositor. Each funding outpoint is recorded in a consensus set the first time it mints, so one deposit mints once. The open and the preimage-revealing claim both also require the consensus header tip to be fresh. The refund-lock margin is measured against that tip, so if its timestamp is more than four hours behind the block clock the mint is refused until headers catch up; otherwise a stalled relay would let a depositor refund on Bitcoin and still claim. A claim is also refused once the tip plus the reserve's sweep margin, 144 Bitcoin blocks on mainnet, reaches the refund lock, so at the moment of the claim the reserve has that window left.

No signature says whether a mint exists, how big it is, or what backs it. Bitcoin's proof of work decides all of that. The one signature in the flow chooses the recipient: whoever opens the mint must present an ECDSA signature by the deposit's Bitcoin refund key over the funding outpoint and the chosen shielded recipient key, so only the depositor, who set that refund key in the SPV-proven script, can say where the zBTC goes. That shuts a front-running attack in which a third party sees the public deposit proof and opens the mint to themselves, and it adds no authority over issuance. The signature is secp256k1, and ZCrypt consensus is what verifies it. Bitcoin never sees it. A quantum forgery of the refund key redirects the mint to a recipient the depositor did not choose. Since no key set can ever commit to a reserve-key authority, the door only swings one way: zBTC cannot be rotated into discretionary issuance.

Reserve custody is a keyholder assumption. Consensus constrains the mint proof, so zBTC is not created without an SPV proof that a funding output paid the pinned key, but it does not constrain what that key does afterwards, and it does not constrain the reserve keyholder. The accumulated BTC is spendable by that key alone, and losing or leaking it leaves the zBTC supply intact while stranding redemption. The reserve key can be rotated on chain. A rotation transaction signed by the current reserve key over the chain identifier, the asset, a monotonic epoch, and both keys retires the old key and pins the new one. Deposits already mined to a retired key, plus a short grace of blocks, still mint and can still be re-anchored, so nothing in flight is stranded, and every later deposit has to pay the new key, which wallets read from the asset's registry entry. Rotation only ever moves between reserve keys, never to a signer set, so zBTC stays SPV-gated. And because consensus only checks that deposits pay the pinned public key, that key can itself be held by threshold ECDSA among independent operators without any protocol change. How the reserve key is held is a deployment decision, and a deployment should say what it is.

## 10.3 Maturity and Reorg Safety

An SPV proof shows a deposit was confirmed, but Bitcoin only offers probabilistic finality, and a deep enough reorganization can orphan a confirmed block. A minted zBTC coin enters the shielded pool and becomes unlinkable, so there is no clawing it back afterwards; the peg has to stop an unbacked coin from ever being spent in the first place. The design borrows Bitcoin's coinbase-maturity rule. A newly minted coin is held immature, outside the spendable pool and therefore outside the anonymity set, until its deposit is buried past reorganization risk. Every block, consensus sweeps the immature set: a deposit buried to its maturity depth becomes a spendable coin, and a deposit whose block was orphaned is handled as described below.

Maturity depth scales with value. Orphaning a deposit costs, at minimum, its depth times the block reward in forgone honest mining, on top of the majority hashpower it takes, so the attack only pays when the unbacked value exceeds that. A flat depth would make a small shield wait as long as a large one. Instead the required depth grows with the minted value, floored at the mint's minimum confirmations and capped near Bitcoin's hundred-block coinbase maturity. Small transfers mature in about an hour. Large ones wait deep, which is exactly where an attacker would bother.

Orphaning is not treated as an immediate void, because a benign reorganization usually re-mines the same deposit into a different block and only a genuine double-spend drops it. On orphaning, consensus opens a grace

window during which a relayer may re-anchor the immature mint by submitting a fresh inclusion proof for the same deposit txid on the new best chain; the anchor moves forward and maturation resumes. A deposit that never reappears fails to re-anchor, and when the grace runs out its immature coin is voided and the supply burned back. All of this is a function of the header chain in the state root, so validators agree on which coins mature, re-anchor, or void.

### 10.4 The Atomic Swap

Minting and redemption are two hash-time-locked contracts in the classic atomic-swap shape [23], sharing one untagged SHA-256 hashlock so that a single preimage satisfies Bitcoin script's OP_SHA256 and ZCrypt's escrow check alike. On Bitcoin the contract is a standard P2WSH output: preimage plus signature to the claimant, or a signature alone to the refunder after an absolute OP_CHECKLOCKTIMEVERIFY height. On ZCrypt a consensus escrow locks a public (value, asset) pair under a hashlock and a block-height timeout. Settlement coins use deterministic public blinds, so the swap amount is public, which an atomic swap needs, but the settled coins go back into the private pool.

**BTC → zBTC.** The depositor picks a 32-byte random preimage (the Bitcoin script pins the size with OP_SIZE, matching ZCrypt's escrow rule), funds the Bitcoin HTLC, claimable by the reserve with the preimage and refundable to the depositor after its locktime, waits for confirmation, relays the intervening headers, and submits the mint proof. Consensus opens the mint escrow, and when the depositor claims with the preimage it mints immature zBTC. The reserve reads the revealed preimage off ZCrypt and sweeps the BTC. The zBTC matures once the deposit is deep enough. If the proof is never submitted the depositor refunds on Bitcoin; if the claim never comes the escrow expires and nothing is minted. Maturity checks that the funding transaction is still included on the best header chain, at its value-scaled depth. It does not check that the reserve's sweep of that output has confirmed. If the reserve is offline, or Bitcoin fees move the sweep past the refund lock, the depositor refunds the BTC and the immature coin still matures, because the funding transaction remains in its block. The sweep margin buys the reserve a window. Solvency of the peg depends on the reserve using it. The operator is also told to wait until the deposit is past maturity plus the reorg grace before sweeping, so that a voided mint does not strand BTC already taken; that advice and the refund lock are the two ends of the same window.

**zBTC → BTC.** The same shape with the roles swapped. The redeemer locks zBTC in a burn escrow, a liquidity provider funds a Bitcoin HTLC claimable by the redeemer, the redeemer's Bitcoin claim reveals the preimage, and the provider uses it to finalize the burn and reduce tracked supply. Every failure path refunds the escrow. The chain burns the zBTC and does not pay the provider; whatever reimburses the provider is an arrangement off the protocol. The provider who already holds the reserve's BTC is the one for whom that arrangement is internal. In the reference deployment that holder is the single reserve key the shipped wallet stores.

Issuance of zBTC requires an SPV proof of a reserve-bound funding output, and the protocol-pinned cap bounds the supply counter. Backing of that supply also requires the reserve to sweep before the refund lock, which is a liveness assumption about the keyholder, and redemption requires a provider the chain does not pay. Reserve custody, above, is where that assumption sits. What the construction removes is a federation that must sign each mint. What it keeps is one secp256k1 key that can spend the locked coins, a maturity delay, and a dependence on that key being online for both the sweep and the redemption.

## 11 Private In-Pool Swaps

The same escrow machinery lets two parties trade one shielded asset for another without either leaving the pool: peer to peer, no order book, and no pooled liquidity. A timed-out lock returns to its locker, so a stalled counterparty does not take the principal. The taker still holds the usual hash-lock option: after the maker has locked, the taker can claim or walk away for the length of the timeout, and the maker cannot.

A third escrow kind, the swap escrow, locks value out of the pool the way a redemption does, with a shielded spend whose first output is a revealed-value commitment to the burn key, alongside the asset identifier, a hashlock, a timeout, a counterparty key, and a refund key. Claiming with the preimage before the timeout re-materializes the value as a pool coin for the counterparty; expiry re-materializes it for the locker. Supply counters do not move either way. A swap changes ownership, not issuance, and needs no authority.

Two parties who have agreed a price off chain (a request-for-quote exchange between wallets; the chain never sees or checks the ratio) proceed like this. The taker picks a random preimage and locks their side, say $x$ of asset $A$, claimable by the maker. The maker sees the hashlock on chain and locks $y$ of asset $B$ under the same hashlock, claimable by the taker. The maker's lock carries the shorter timeout, and the taker's lock expires a margin of blocks later, since the taker is the one who reveals the preimage and the maker must still have time to claim after that reveal; a maker wallet refuses any quote that does not respect this order. The taker claims the maker's escrow, revealing the preimage, and the maker claims the taker's with it. If either side stalls, both escrows expire back to their lockers. It is the Bitcoin swap of §10 with both legs on one chain, and it settles in a handful of blocks.

Privacy composes the same way it does for the bridge. The amounts and the hashlock are public, because an atomic swap needs them to be, but which coins funded each lock hides behind the usual whole-pool membership proof, change comes back as an ordinary commitment with an encrypted note, and the coins each side receives are standard pool coins whose later spends hide among everything else. Each swap lock burns 0.01 ZCRYPT in a separate gas proof, and the locked asset is not charged. A redemption burn-lock still pays its fee in the asset being redeemed.

## 12 The Wallet

The reference wallet is a desktop application with the full prover embedded in its backend. No third party ever sees a witness.

**Proving and sending.** The wallet keeps an incrementally synced copy of the commitment-tree leaves, rebuilds the tree locally, and derives Merkle paths for its own coins. A payment selects up to two coins, builds the gas note and the main transaction, proves both (about 0.22 s on a laptop, §6.2), runs the auditability VM as a last cross-check, and broadcasts the bundle. Change always goes to a fresh stealth sub-address, and every output slot carries a fixed-size note, real or cover, so the delivery mode cannot be fingerprinted.

**Receiving.** Scanning fetches only the blocks that contain transactions and trial-decrypts every note against the account key, the stealth change accounts (with a gap limit), and any imported stealth seeds. Stealth payments arrive as an out-of-band note string. Bearer coins are claimed and immediately swept to a fresh secret.

**Spent status without queries.** Asking a node "is nullifier $X$ spent?" would tell it which coins are yours, so the reference wallet never asks. It downloads every transaction-bearing block, proofs included, and matches the published nullifiers against those of its own coins locally. A coin is spent when its nullifier has shown up in a block the wallet has seen, and all the node learns is that someone did a full-block sync. At roughly 2 MB a transfer, that sync is a desktop workload. Nodes also serve authenticated inclusion and non-inclusion proofs against the registry root for anyone who prefers a point query. Trusting such a proof completely, so that a node could not answer against a registry root consensus never committed, would need that root tied to the block's application hash through validator-signed headers, and that light-client step is one the reference wallet does not yet perform.

**Network privacy and custody.** Node RPC can go over Tor, which is on by default and fails closed if misconfigured. The mnemonic is encrypted at rest under an Argon2id-derived key [24] (64 MiB, 3 passes), and the coin database under a key derived from the seed. The wallet exposes the whole product: balances per asset, the three payment instruments, view-key export, reward claims, asset registration and minting, and both directions of the bridge, with automatic recovery of swaps that were interrupted partway.

## 13 Privacy Analysis

What someone watching the chain sees, action by action:

- **Transfer:** two nullifiers, two commitments, two uniform ciphertexts, the constant 0.01 ZCRYPT gas burn, an anchor root, a proof. No amounts, no asset type, no participants, no arity, and no link from the nullifiers back to any earlier commitment.
- **Reward claim:** amount and recipient shield key. Rewards are public income, so this is intended. The minted coins use deterministic blinds derived from those public values, which makes them identifiable when they are born; they become anonymous once spent, because the spend hides among the whole pool.
- **Bond / delegate / unbond:** the stake and the validator identity, public on purpose, with the source coins anonymous.
- **Bridge or in-pool swap:** the amounts, the escrow parameters, and the claimant's shield key. A bridge deposit also publishes the raw Bitcoin funding transaction, which ties the depositor's Bitcoin history to that shield key. The settlement coin's opening follows from these public values, so it too is identifiable at birth and anonymous only once spent, and a user who reuses one shield key across bridge operations links them all together. That is why the reference wallet derives a fresh key per operation. The coins that funded each leg stay anonymous.
- **Asset registration or authority rotation:** ticker, decimals, cap, authority commitment and epoch. Never a signer's secret.

The metadata that leaks is small for transfers and larger at the edges, where an amount has to be public for consensus to enforce a rule about it (stakes, issuance, escrow amounts, supply caps). Nothing else is exposed. Timing and network-origin metadata are dealt with by Tor by default, a uniform transaction shape, cover notes, and the absence of any transparent value layer that would sort users into pools with different levels of privacy.

The anonymity set is a direct consequence of the membership proof. Because membership is proven against the whole commitment tree, every spend hides among every coin in history, spent or not. The tree currently has $2^{32}$ slots, which is a circuit parameter and not an assumption of the analysis (§5.4 covers capacity). There is no decoy-selection distribution to skew and no ring-intersection attack to run.

## 14 Security Considerations

**Soundness.** Value integrity rests on the STARK, on the hash, and on consensus being correct. The STARK's conjectured list-decoding figure is 244 bits and is the same in every build; the Johnson bound the node checks is 132 bits (§3.2). Poseidon2 preimage resistance is what backs spend authority and the burn key, and those searches are eight lanes wide. Commitment binding is the compression of §3.1: about $2^{248}$ to find a preimage of an eight-limb digest and about $2^{124}$ to find a collision, in the ideal-permutation model. A soundness failure that creates a coin inside the pool is not visible in the issuance digest. That digest proves the foundation vesting and the epoch rewards followed the schedule. It does not prove the pool contains only those coins. zBTC has a partial check of this kind: its supply counter increments on mint and decrements when a redemption burn finalizes, so value that exits can be compared with value that entered. The native coin never exits, and it has no such counter. An inflation that stays in the pool does not show up in either one.

**Encoding discipline.** A field element has one byte encoding anywhere it matters. Transaction parsing rejects non-canonical limbs, and nullifier registry keys are computed over reduced forms, which rules out the double-spend that respends a coin under a second encoding of the same nullifier.

**Replay and binding.** A proof binds a chain-domain tag, so it cannot be replayed on another chain; an anchor from a bounded window, so it cannot be built against ancient state; a transaction-binding hash over that proof's ciphertexts, commitments, fee, and proof bytes, stored as the nullifier leaf; and, for stake pledges and escrow locks, the plaintext parameters of the transaction around it, so that proof cannot be re-wrapped under someone else's header. The gas proof of a transfer, a swap lock, a registration, or an escrow open commits its auxiliary digest to the transaction it pays for (§5.7). Phantom nullifiers are derived from each real input's secret coin tag, so they never collide across transactions even if a spend secret is reused, and an observer who sees only the published nullifier cannot recompute the padding slot. A repeated commitment is rejected at the tree (§4). The nullifier still does not contain the leaf position.

**Consensus economics.** Safety violations are punished by evidence-triggered slashing of 5% of the offender's own stake and of each delegation, with the remaining 95% force-unbonded, and by permanent tombstoning. Delegators share that 5% and have no way to exit ahead of it. Once the sentinel (§7.4) has retired, voting power comes only from stake that left the pool and sits in the staking ledger. Unbonding returns that stake after 10 days, so holding power costs the lock and, on equivocation, the 5% slash. It does not destroy the stake. While the sentinel is active the foundation node holds a disclosed, state-root-committed supermajority, and that is the trust assumption of the bootstrap phase.

**Issuance authority.** Mint authorizations are ML-DSA-65 threshold signatures over the complete escrow parameters under a protocol context string, so one authorization fits one issuance, cannot double as a rotation (rotations carry a different in-message tag), and cannot be replayed, because the deterministic escrow identifier collides. Rotations are bound to a per-asset epoch in the state root, so a superseded key set keeps no power. A malformed authorization bundle (out-of-range or duplicate signer indices, undecodable keys or signatures, trailing bytes) is rejected whole.

**Quantum posture.** Spends use no discrete-logarithm assumption. Proofs and spend authority are hash-based, note confidentiality is lattice-based, and recorded ciphertexts are safe against harvest-now-decrypt-later. What classical cryptography remains is not all peripheral. ed25519 on validator consensus keys is an availability matter. secp256k1 on the Bitcoin script is Bitcoin's signature, and a quantum attacker against Bitcoin is a fact about Bitcoin. The secp256k1 signature that chooses a zBTC mint's recipient, and the one that rotates the reserve key, are checked by ZCrypt consensus and by nobody on Bitcoin. A quantum forgery of those keys redirects mints or rotates the reserve inside ZCrypt's own rules.

**Assurance.** The circuit is differentially tested against an independent wide-trace implementation and against the transparent VM on sampled witnesses, valid and adversarial; wallets re-run the VM before broadcasting, which constrains honest provers; nodes reproduce their state root from persisted snapshots and refuse to start if they cannot. The check consensus actually applies to a transfer is the AIR. The seven-opcode program is the specification and the prover-side cross-check. It is not in the path that accepts a proof. The emission arithmetic is in the state root and is auditable, with the limit in the soundness paragraph above: it audits the schedule, not the pool. Zero knowledge of published proofs is the unproved hiding layer of §3.2.

**Machine-checked verification.** Differential tests compare the circuit to its specification on sampled witnesses. A companion effort lifts part of that to machine-checked proofs in Lean 4 [30], kernel-verified with no sorry. Two things are established so far. First, the transfer's value-integrity specification (the transparent VM of §6) and the arithmetic at the core of the circuit are formalized and proven directly: an accepted transfer creates no value, since total outputs never exceed total inputs, and the multi-limb range-checked carry that reconciles field arithmetic with integer arithmetic provably cannot wrap, so conservation in the field is conservation over the integers. The coin-layer file does not assume the hash is injective. No function from sixteen field elements to eight is injective, so that hypothesis would be false of the compression and every consequence of it would

hold vacuously. Each coin-layer result is a reduction. If two distinct openings give the same commitment, or a commitment equals a nullifier or a public key, the proof returns the two hash inputs that collide. Breaking the property requires finding a collision. The nullifier of one spend is a function of that spend, and that fact needs no assumption. Collision resistance of the compression is the computational assumption, and the file does not discharge it. Second, so that the proofs are about the source rather than a hand transcription of it, the value-integrity core of the VM is mechanically extracted from its Rust into Lean with Charon and Aeneas [31], the same Rust-to-Lean pipeline the Ethereum Foundation's zkEVM verification project has applied to Plonky3 primitives [32], and the no-inflation theorem is proved again against that machine-generated model.

Here is the boundary. What is verified unconditionally is the transfer specification's arithmetic: outputs do not exceed inputs, and the range-checked carry does not wrap. The coin-layer results are that collision reduction. They are not a proof that the concrete hash is collision resistant. The FRI soundness of §3.2 and the lattice assumptions of §3.3 are further assumptions the proofs do not discharge. What is not yet machine-checked is the full constraint system, the AIR itself. Showing that the deployed constraints enforce exactly the specification, with no under-constrained witness, needs a formal model of AIR and STARK semantics, and that is future work; until then the differential tests are the bridge between specification and circuit, and those tests sample witnesses rather than columns. None of this replaces an external audit. The development uses the Lean core library plus the collision reduction at the coin layer, and it rebuilds from the published artifact with one command.

# 15 Deployment Models

What this paper describes, a shielded pool with hash-based proofs on the spend path, lattice note encryption, threshold lattice signatures on issuance, and selective disclosure, is a neutral protocol. The bridge still checks secp256k1 signatures inside ZCrypt's own rules (§10). It could be the base for private digital cash, or the technical infrastructure for a central bank digital currency. Neither is advocated here.

## 15.1 Central Bank Digital Currency

CBDC design is caught between two demands. Monetary authorities need control over issuance and compliance; citizens have a legitimate expectation of transactional privacy, which is what physical cash has always given them. A recent BIS working paper [25] puts it this way: "better technological solutions offering hard privacy need to emerge." The primitives here happen to fit. The threshold mint authority (§9.1) is an exclusive, consensus-enforced issuance mechanism that spreads signing power across $M$-of-$N$ institutions and can be rotated without redeploying anything. The shielded pool keeps ordinary commerce private. The nullifier registry gives the global double-spend prevention a monetary authority needs. The consensus layer can be run by the issuer or by licensed delegates, and the transfer layer stays private regardless.

A CBDC on these primitives would get privacy-preserving compliance through tiered disclosure, the inverse of designs that start from full visibility and try to carve out a little privacy. Routine transactions are fully private, like cash. The issuer sees aggregate nullifier volume, a privacy-preserving proxy for velocity, and not individual transactions. Where policy demands more, disclosure is additive and scoped to keys: view keys (§4.3) let a wallet show its incoming flows, amounts and asset types, to a designated auditor or, under a court order, to a financial intelligence unit, without giving up the ability to spend. Privacy is the base layer, and compliance goes on top of it.

There is a surveillance risk, and it should be said out loud. Privacy infrastructure deployed by a state can be repurposed. Threshold reporting is a policy surface that can be lowered to zero, judicial disclosure depends on courts that may be under political pressure, and a mandate to hand over view keys at issuance would turn a privacy tool into an escrow system. As a matter of architecture, the zero-knowledge proof system contains no compliance hooks, trapdoors, or key escrow. Privacy is a property of the cryptography and not a setting that can be switched off later.

Central banks work on generational timescales, which makes the quantum question urgent for them in a way it is not yet for most. A CBDC on elliptic-curve cryptography faces store-now-decrypt-later: capture the traffic today, decrypt it when the machines arrive. Signature-based CBDC designs also run into a quantum-safe performance wall; BIS Project Tourbillon [26] measured roughly a 5× latency penalty and a 200× throughput reduction when it moved blind-signature eCash [27] to post-quantum primitives. The proof-based design here does not have that problem. The spend path is hash-based from genesis, and a pool payment has no signature. The bridge still checks secp256k1 signatures inside ZCrypt's own rules, for the mint recipient and for reserve-key rotation (§10). None of this is advocacy. Whether these properties are wanted in a CBDC is for monetary authorities and the publics they answer to.

## 15.2 Private Digital Cash

Deployed as an open network, ZCrypt's instruments can move without an account. A bearer coin is the 112-byte opening of a pool coin. Whoever holds those bytes can spend it, and so can anyone who kept a copy. Handing the string over does not take it away from the giver. It is worth what the receiver's trust in that giver is worth, until the receiver sweeps it to a fresh secret the giver does not have. A stealth payment is an encrypted note,

and it leaves nothing on chain that a view key can open. Finality in either case is a sweep or a claim against the network. Until that happens the string is a copyable secret, not a banknote: a banknote spent from the giver's hand is no longer in it.

This is what Chaum described in 1983 [28, 29]: ecash as a privacy-preserving medium of exchange, here without his dependence on a central mint. Remittances settle in seconds over the channels people already use, with no processor taking a cut or keeping a record. The same properties serve organisations that need confidentiality. A corporate treasury gets post-quantum protection and privacy from competitors watching the chain; funds can move without signalling intent to a market; an aid organisation can disburse without building a ledger that a hostile actor could use to find the recipients.

## 16 Conclusion

ZCrypt makes one structural bet: a ledger is most private, and most fungible, when privacy is the substrate rather than a feature. One pool holds all the value, a shielded transfer moves pool coins, and the anonymity set is the history of the system. Around that core the rules are transparent, as bytecode specifications, an auditable emission schedule, and a state root that commits to every ledger, while the activity is hidden. The spend path does not build on a discrete-log assumption: hash-based proofs with no ceremony, ownership by preimage with no signature on the spend, lattice-based confidentiality with no harvestable ciphertexts. The bridge still verifies secp256k1 signatures inside ZCrypt's own rules (§10). Stake bonds, reward issuance, and asset creation pass into the same pool through the gates this paper has described, with the binding, soundness, and custody limits of §14. The rules, the emission schedule, and the registry are committed in the state root. The pool's contents are not a transparent ledger: a failure that creates a coin inside the pool does not appear in the issuance digest (§14). What an observer can audit is that the published rules were followed. Who paid whom stays hidden.

## Acknowledgments

Thanks to Vasily Pozdyshev for his feedback on institutional adoption, Jameson Lopp for his feedback on non-inclusion proofs, and the zero-knowledge and post-quantum cryptography communities for advancing the primitives used here.

## References

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.

[2] N. van Saberhagen, "CryptoNote v2.0," October 2013.

[3] D. Hopwood, S. Bowe, T. Hornby, N. Wilcox, "Zcash Protocol Specification," 2016.

[4] Zcash, "Zcash Protocol Specification," version 2024.5.1, Orchard note commitments and the in-circuit Merkle path, https://zips.z.cash/protocol/protocol.pdf.

[5] L. Parker, "Full-Chain Membership Proofs," Monero, April 2024, https://www.getmonero.org/2024/04/27/fcmps.html.

[6] A. Szepieniec and T. Værge, "Neptune," April 2021, https://neptune.cash/whitepaper. The live network encrypts notes under lattice keys (https://neptune.cash).

[7] S. Bowe, J. Grigg, D. Hopwood, "Recursive Proof Composition without a Trusted Setup," IACR ePrint 2019/1021.

[8] Penumbra Labs, "The Penumbra Protocol," protocol specification, https://protocol.penumbra.zone, accessed September 2026.

[9] C. Goes, A. Sun Yin, A. Brink, "Namada: Interchain Asset-Agnostic Privacy," 2022.

[10] L. Grassi, D. Khovratovich, M. Schofnegger, "Poseidon2: A Faster Version of the Poseidon Hash Function," AFRICACRYPT 2023.

[11] National Institute of Standards and Technology, "FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," 2015.

[12] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev, "Scalable, Transparent, and Post-Quantum Secure Computational Integrity," IACR ePrint 2018/046.

[13] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev, "Fast Reed–Solomon Interactive Oracle Proofs of Proximity," ICALP 2018.

[14] E. Crites, A. Stewart, "On Reed–Solomon Proximity Gaps Conjectures," IACR ePrint 2025/2046.

[15] Polygon Zero and contributors, "Plonky3: A Toolkit for Polynomial IOPs," https://github.com/Plonky3/Plonky3, 2024.

[16] National Institute of Standards and Technology, "FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard," 2024.

[17] H. Krawczyk, P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, 2010.

[18] Y. Nir, A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," RFC 8439, 2018.

[19] M. Palatinus, P. Rusnak, A. Voisine, S. Bowe, "BIP-39: Mnemonic Code for Generating Deterministic Keys," 2013.

[20] P. Wuille, "BIP-350: Bech32m Format for v1+ Witness Addresses," 2020.

[21] E. Buchman, J. Kwon, Z. Milosevic, "The Latest Gossip on BFT Consensus," arXiv:1807.04938; CometBFT, the maintained implementation.

[22] National Institute of Standards and Technology, "FIPS 204: Module-Lattice-Based Digital Signature Standard," 2024.

[23] T. Nolan, "Alt Chains and Atomic Transfers," bitcointalk.org, https://bitcointalk.org/index.php?topic=193281.0, May 2013.

[24] A. Biryukov, D. Dinu, D. Khovratovich, S. Josefsson, "Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications," RFC 9106, 2021.

[25] R. Auer, R. Böhme, J. Clark, D. Demirag, "Privacy-Enhancing Technologies for Digital Payments: Mapping the Landscape," BIS Working Papers No. 1242, January 2025.

[26] BIS Innovation Hub Swiss Centre, "Project Tourbillon: Exploring Privacy, Security and Scalability for CBDCs," Bank for International Settlements, November 2023.

[27] D. Chaum, T. Moser, "eCash 2.0: Inalienably Private and Quantum-Resistant to Counterfeiting," 2022.

[28] D. Chaum, "Blind Signatures for Untraceable Payments," Advances in Cryptology: CRYPTO '82, 1983.

[29] D. Chaum, A. Fiat, M. Naor, "Untraceable Electronic Cash," Advances in Cryptology: CRYPTO '88, 1990.

[30] L. de Moura, S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," CADE 28, 2021.

[31] S. Ho, J. Protzenko, "Aeneas: Rust Verification by Functional Translation," Proc. ACM Program. Lang. (ICFP), 2022.

[32] N. Klaus, J. Conejero, P. Tolmach, "A Rust-to-Lean Verification Pipeline with AI Provers: An Experience Report," arXiv:2605.30106, 2026.